



MediTracker

A Role-Based Mobile Application for Medication Adherence and Caregiver

Monitoring



Project submitted in the context of the course

IN448 — Project Management

Prepared by

Mohamad Khatib

Instructors:

Dr. Rami Tawil, Dr. Ali Jaber

Lebanese University

Faculty of Science I

Department of Computer Science

2025 – 2026

Acknowledgment

First and foremost, I would like to express my deepest gratitude to my course instructors for their guidance, constructive feedback, and invaluable academic oversight throughout the duration of the IN448 — Project management course.

I am equally grateful to the faculty members of the department of Computer Science at the Lebanese university for providing the foundational software engineering knowledge and academic environment necessary to conceive and execute this project.

— Mohamad Khatib
2026

Abstract

Patient non-adherence to prescribed medical routines represent a critical healthcare challenge, causing preventable disease progression and increased tracking burdens on families. MediTracker is a full-stack, cross-platform mobile application built to address this problem directly: it gives patients a structured way to schedule and log their medications, and gives linked caregivers visibility into that routine without requiring the patient to manually report it.

The system follows a three-tier architecture. The presentation tier is a React Native mobile client with role-aware navigation for patients and caregivers. The business logic tier is a Node.js and Express REST API that enforces authentication, role-based authorization and a caregiver-patient relationship state machine. The data tier is a MongoDB database accessed through Mongoose schemas modeling users, medications, adherence logs, and caregiver links.

Three aspects of the implementation are highlighted in this report. First, the data model was designed to capture realistic medication schedules — daily, specific days, fixed intervals, or as-needed — together with optional inventory tracking and dosing instructions. Second, a background Cron service evaluates due and overdue doses every minute, automatically marking unlogged doses as missed after a grace window and dispatching push notifications to patients and their linked caregivers through the Expo push notification service. Third, the caregiver-patient relationship is modeled as an explicit state machine (pending, approved, rejected, revoked) that either party can initiate, accept, or end, with the backend enforcing that a caregiver can only view a patient's data once an approved link exists.

The resulting application delivers the medication tracking and caregiver monitoring functionality defined in the project's scope: secure registration and login, full CRUD on medications, dose logging with automatic missed-dose detection, a bidirectional linking workflow between patient and caregivers, and real-time push notifications, all deployed against a live backend and a cloud-hosted database.

Table of Content

Acknowledgment	2
Abstract	3
Table of Content	4
Table of Figures	5
Table of Tables	6
Chapter I — Introduction	7
I.1. The Business Domain	7
I.2. About the Modeled Application	8
I.3. Analysis of the Existing Similar Apps	9
I.4. Plan of the Document	10
Chapter II — Requirement Analysis and Specification	11
II.1. Introduction	11
II.2 Overall Description	12
II.3. External Interface Requirements	14
II.4. System Features	16
II.5. Other Non-Functional Requirements	22
II.6. Use Case Diagram and Descriptions	24
Chapter III — Application Conception	27
III.1. System Architecture	27
III.2. Database / Data Model Design	28
III.3. API Design	29
III.4. Sequence Diagrams	30
III.5. State Chart Diagram	31
Chapter IV — Development, Testing and Deployment	32

IV.1. Development Methodology -----	32
IV.2. Technology stack justification -----	33
IV.3. Implementation Highlights -----	34
IV.4. Challenges Encountered-----	35
IV.5. Testing and Debugging Approach-----	36
IV.6 Deployment-----	37
IV.7. Conclusion and Future Work -----	37

Table of Figures

Figure 1- The scale of medication non-adherence among chronic disease patients.

Figure 2- UML use case diagram for MediTracker.

Figure 3- Three-tier system architecture of MediTracker.

Figure 4- UML class diagram of the core data model.

Figure 5- Sequence diagram - log medication dose with caregiver alert

Figure 6- Sequence diagram — Caregiver-Patient link invitation

Figure 7- State chart diagram — Caregiver link status lifecycle

Table of Tables

Table 1- comparison of existing similar medication-reminder applications.

Table 2- User class characteristics for the two roles supported by MediTracker

Table 3- Summary of external software interfaces consumed by the backend

Table 4a- Functional requirements — Authentication and Account Management.

Table 4b- Functional requirements — Medication Management.

Table 4c- Functional requirements — Daily Schedule and Dose Logging

Table 4d- Functional requirements — Automated Reminders and Missed-Dose Detection.

Table 4e- Functional requirements — Adherence History.

Table 4f- Functional requirements — Caregiver-Patient Linking.

Table 5a- Non-Functional Requirements — Performance.

Table 5b- Non-Functional Requirements — Security.

Table 5c- Non-Functional Requirements — Usability.

Table 5d- Non-Functional Requirements — Reliability and Availability.

Table 5e- Non-Functional Requirements — Maintainability.

Table 6a- Use case description — Log Dose

Table 6b- Use case description — Send Caregiver Link invitation.

Table 7-Mongoose collections and their purpose.

Table 8-REST API endpoint summary

Table 9- Technology stack and justification.

Chapter I — Introduction

I.1. The Business Domain

Medication non-adherence is one of the most persistent and costly problems in chronic disease management. Roughly half of patients with long term conditions such as hypertension or diabetes do not take their medication as prescribed, and this single behavior is associated with an estimated 125,000 preventable deaths and between \$100 billion and \$300 billion in avoidable healthcare spending in the United States every year, along with a meaningful share of hospital admissions.

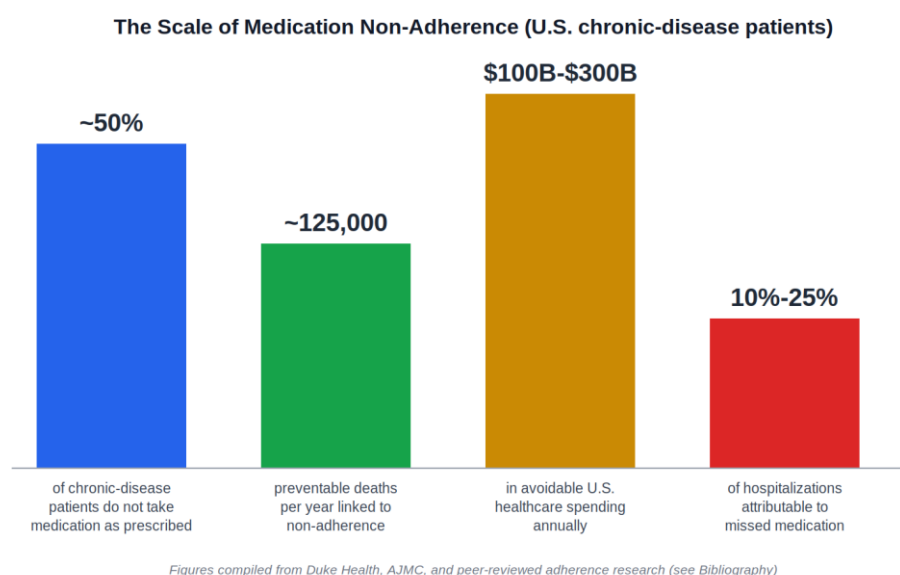


Figure 1- The scale of medication non-adherence among chronic disease patients.

The problem is not purely forgetfulness. It is also a visibility problem: when a patient lives alone or a family member cannot be present at every dose time, there is usually no reliable way for a caregiver to know whether a treatment plan is actually being followed until a missed dose has already caused a setback. This is the specific gap MediTracker is built to close: it does not just remind a patient to take their medication, it gives a linked caregiver a passive, real-time view of whether doses are being taken, skipped, or missed, without requiring the patient to actively report anything.

I.2. About the Modeled Application

Purpose

MediTracker is a cross-platform mobile application that lets a patient register a set of medication with flexible scheduling rules, log each dose as taken or skipped, and review a history of that adherence over time. A second user role, the caregiver, can request to be linked to a patient and, once that link is approved, gains read access to the patient's medication list and adherence history, and receives push notifications when a dose is taken, skipped, or automatically detected as missed.

User Personas

- **Patient** — an individual managing one or more ongoing medications who want a single place to see what is due today, log doses with one tap, and keep a history without paper notes or memory alone,
- **Caregiver** — a family member or a guardian who is not necessarily present at every dose time but wants timely, low effort visibility into a patient's adherence, and to be alerted automatically if a dose is missed.

Product Perspective

MediTracker is a new, self-contained product rather than an extension of an existing system. It is composed of a mobile client built with React Native and Expo that talks to a single REST API built with Node.js, Express and MongoDB through mongoose over HTTPS. Both patient and caregiver roles are served by the same mobile codebase, with the interface and set of available screens determined by the role stored on authenticated user's account.

I.3. Analysis of the Existing Similar Apps

Several mobile applications already address medication reminders and adherence tracking. The most relevant points of comparison are Medisafe and MyTherapy, two of the most widely used apps in this category as of 2026.

- **Medisafe** — It offers a caregiver mode “MediFriend” that notifies a chosen contact on a missed dose, but its free tier limits the number of medications that can be tracked and gates drug-interaction warnings behind a paid subscription.
- **MyTherapy** — combines medication reminders with a broader personal health journal and can generate adherence reports for a doctor’s visit.

The table below summarizes how these two applications compare against the feature set actually implemented in MediTracker.

Feature	Medisafe	MyTherapy	MediTracker
Flexible dosing schedules (daily, specific days, interval, PNR)	Partial	partial	yes
Free, unlimited medications	No (paid tier)	yes	yes
Dedicated caregiver role with approval-based linking	yes	no	Yes
Automatic missed dose detection	No	No	Yes
Real-time push alert to caregiver on missed/taken/skipped doses	Partial	No	Yes
Inventory / refill tracking	Yes	No	Yes
General health journal	No	Yes	No
Drug interaction database	Paid tier	No	No

Table 1- comparison of existing similar medication-reminder applications.

As the comparison shows, most existing tools treat caregiver as a secondary, notification-only contact rather than a first-class user with their own dashboard, and none of the surveyed applications automatically infers a missed dose purely from elapsed time, they rely on the patient to mark a dose as skipped or simply do not raise an alert at all. MediTracker's caregiver role is a fully modeled user type with its own authentication, dashboard, and explicit approval workflow, and its backend Cron engine actively detects a missed dose 30 minutes after it was due and notifies both the patient and any linked caregiver without requiring either to take action first. In exchange, MediTracker deliberately does not attempt to be a general health journal or a drug-interaction database; its scope is kept to adherence tracking and caregiver monitoring, which is the gap identified in section I.1.

I.4. Plan of the Document

The remainder of this document is organized as follows. Chapter II extends the IEEE software requirements specification template to describe MediTracker's functional and non-functional requirements, the requirements-gathering process that was followed, and the use cases derived from it. Chapter III presents the application's conception through UML diagrams: a class diagram of the core data model, sequence diagram of the two most significant interaction flows, and a state chart diagram of the caregiver-patient relationship lifecycle. Chapter IV covers the development process itself: the methodology followed, the technology stack, the challenges encountered, and how testing, debugging, and deployment were handled, before concluding with a reflection on the finished product and directions for future work.

Chapter II — Requirement Analysis and Specification

(Extension of the SRS document provided by IEEE)

II.1. Introduction

1.1. Purpose

This document specifies the software requirements for MediTracker, version 1.0, a cross-platform mobile application for medication adherence tracking and caregiver monitoring. It covers the complete system as delivered: the React Native / Expo mobile client and the Node.js / Express / MongoDB backend that supports it. There is no separate sub-system excluded from this specification.

1.2. Document conventions

Functional requirements are identified with the prefix REQ- followed by a sequential number, grouped under the system feature they belong to. Priority is expressed using the MoSCoW method (Must have, Should have, Could have, Won't have this time) rather than a numeric scale, since this matches how the project's own backlog was actually managed. Higher-level feature priority is inherited by the functional requirements listed beneath it unless a requirement explicitly states a different priority.

1.3. Intended Audience and Reading Suggestions

This document is intended for the course instructors evaluating the project, and secondarily for any future developer who picks up the codebase. A reader interested only in what the system does should read sections II.2 and II.4. A reader interested in why specific design decisions were made should continue into chapter III (Application conception) and chapter IV (Development)

1.4. Product Scope

MediTracker's scope is limited to: (1) secure registration and authentication for two user roles, patient and caregiver; (2) full create/read/update/archive management of a patient's medications, including flexible scheduling; (3) logging of doses as taken or skipped, with automatic detection of missed doses; (4) a bidirectional invitation system that links a caregiver to a patient with explicit approval; and (5) push notifications driven by a backend

scheduler. Features that were discussed during initial project scoping but are not part of the delivered system — such as drug interaction lookups against an external pharmaceutical database, machine learning based adherence prediction, and offline-first data entry — are intentionally excluded from this specification and are instead discussed as future work in section IV.1.

1.5. References

- IEEE Recommended Practice for Software Requirements Specifications, IEEE std 830-1998 (template structure followed in this chapter)
- MediTracker source code used as the ground truth for every requirement listed below.

II.2 Overall Description

2.1. Product Perspective

MediTracker is a new, self-contained product. Figure 1 (chapter 1) and the architecture diagram in chapter III show its place in the wider system landscape: a single mobile client communicates over HTTPS with a single REST API, which is the sole point of access to a MongoDB Atlas cloud database. There is no legacy system being replaced and no other subsystem that MediTracker must interoperate with.

2.2. Product Functions

At a high level, the delivered system lets a user:

- Register an account as either a patient or a caregiver, and login securely.
- (Patient) Add, edit and archive medications with a name, dosage, type, schedule, optional inventory tracking, and optional doctor information.
- (Patient) View a calendar-driven daily schedule of medications due on a selected data, and log each dose as taken or skipped.
- (Patient) View a full chronological history of logged and automatically detected doses.
- (Patient or caregiver) Send a link invitation to the other role by email or username, and approve, reject, or revoke that link.
- (Caregiver) View the medication list and adherence history of every patient with whom an approved link exists.

- (Patient or Caregiver) Receive push notifications for dose-due reminders (Patient), does-taken/skipped confirmations (Caregiver), and missed-dose alerts (Both roles).
- (Patient or Caregiver) View and edit their own profile information, and change their passwords.

2.3. User Classes and Characteristics

User Class	Characteristics	Frequency of Use
Patient	Manages their own medications; primary data entry role; expected to have low-to-moderate technical proficiency, so the interface favors large tap targets and minimal required fields	Multiple times daily (dose logging)
Caregiver	Read-access role; views one or more linked patients' data; does not create medications or logs directly.	Several times per day to a few times per week

Table 2- User class characteristics for the two roles supported by MediTracker

2.4. Operating Environment

- Mobile Client: Android and IOS devices supported by EXPO SDK 56 / React Native 0.85; tested primarily on a development build for Android.
- Backend: Node.js runtime, deployed as a web service on Render; listens over HTTPS.
- Database: MongoDB Atlas, a cloud hosted, fully managed MongoDB cluster, reachable from the backend over a MongoDB connection string.
- Push delivery: Expo's push notification service relays messages to Android devices using Expo push tokens generated by the client.

2.5. Design and Implementation Constrains

- The mobile client must use Expo managed/dev-client workflow since the project depends on Expo modules such as expo-notifications and expo-router.
- Authentication is stateless and token-based (JWT): the backend does not maintain server-side sessions, which constrains every protected route to re-evaluate the token on each request.

- MongoDB's schema-flexible document model is used deliberately for fields such as medication's frequency and inventory, which are stored as nested objects rather than normalized into separate collections, trading some query flexibility for simpler reads in the mobile client.
- The missed dose detection logic runs as an in-process node-cron job inside the same Node.js process as the API; this constrains the backend to run as a single long-lived process rather than a stateless serverless function, since a serverless platform would not reliably keep a per-minute cron alive.

2.6. User Documentation

End-user documentation is limited to in-app affordance (placeholder text, toast messages on error, and a single onboarding welcome screen). No separate user manual or PDF guide is shipped with the product. This report itself, together with the source code comments, serves as the technical and functional documentation of record.

2.7. Assumptions and dependencies

- It is assumed the end user's device has internet access at the moment a dose needs to be logged. The current implementation does not queue actions for later sync if the device is offline (see section IV.1, future considerations).
- The system depends on three external services remaining available: MongoDB Atlas, Render, and Expo's push notification service. An outage in any of the three would degrade or interrupt the corresponding functionality.
- It is assumed that a single MongoDB connection string and a single JWT signing secret, both supplied through environment variables, are sufficient for the deployment. No secret-rotation or multi-region deployment strategy was implemented.

II.3. External Interface Requirements

3.1. User interfaces

The user interface is a set of mobile screens grouped by Expo Router router groups: (auth) for login and registration, (patient) for the patient dashboard, medication form, and history log, (caregiver) for the caregiver dashboard and patient-log detail view, (shared) for profile/settings and the connections screen for caregiver-link. Every authenticated screen

shares a common header component showing the MediTracker brand, the signed-in user's name and role, and a dropdown menu for navigation and sign-out. Role-based layouts (see chapter III) prevents a patient from rendering caregiver screens and vice versa.

3.2. Hardware Interfaces

The application has no direct hardware interface beyond what the host mobile operating system already abstracts: it reads the device's notification permission state and relies on the OS-level push notification channel to display alerts. No Bluetooth, camera, or other sensor integration is part of the current scope.

3.3. Software Interfaces

Software Component	Version / Provider	Interface Purpose
MongoDB	Atlas, accessed via mongoose 9x	Primary data persistence for all four collections.
Expo server SDK	Expo-server-sdk 6.x	Sends push notifications payloads to devices identified by Expo push tokens.
Expo notifications	Expo-notifications	Registers the device for push notifications, requests permissions, and handles tap behavior
node-cron	4.x	Schedules the per-minute background job that checks for due and missed doses
JSON web token	9.x	Issues and verifies signed JWT used to authenticate every protected API request
bcrypt	6.x	One-way hashing of user passwords before persistence

Table 3- Summary of external software interfaces consumed by the backend

3.4 Communications interfaces

The mobile client and the backend communicate exclusively over HTTPS using JSON request and response bodies. Every protected endpoint expects an Authorization: Bearer <token> header carrying a JWT issued at login or registration. Push notifications are delivered through a separate channel: the backend posts a JSON payload to Expo's push API, and Expo relays it to the device over its own delivery mechanism.

II.4. System Features

This section enumerates the functional requirements actually implemented in the MediTracker codebase, grouped by feature area. Each requirement is traceable to a concrete controller, route, or screen in the repository.

4.1. Feature: Authentication and Account Management

Description and Priority: Must have. Without authentication no other feature is reachable, this was the first feature implemented in the project.

Stimulus/Response Sequences: A visitor opens the app -> if a valid token is already stored, they are redirected directly to their role's dashboard, otherwise they see the welcome screen with Login and register options.

ID	Requirement	Priority
REQ-1.1	The system shall allow a new user to register with a name, username, email, password, and a role (patient or caregiver).	Must
REQ-1.2	The system shall reject registration if the email or username is in already in use.	Must
REQ-1.3	The system shall hash passwords with bcrypt before persisting the user document. Plain text passwords shall never be stored.	Must
REQ-1.4	The system shall allow a registered user to log in with their email and password and receive a signed JWT on success	Must

REQ-1.5	The system shall persist the issued token on the device and automatically re-authenticate the user on app launch without requiring login again, until the token expiration or logging out.	Must
REQ-1.6	The system shall reject any request to a protected route that does not present a valid, unexpired JWT.	Must
REQ-1.7	The system shall restrict access to role-specific screens and endpoints so that a patient cannot reach caregiver-only functionalities and vice versa	Must
REQ-1.8	The system shall allow a logged-in user to view and update their profile and to logout, clearing the stored token	Should

Table 4a- Functional requirements — Authentication and Account Management.

4.2. Feature: Medication Management (CRUD)

Description and Priority: Must have. This is the core data entry feature that every other feature depends on.

Stimulus/Response Sequences: A patient taps “Add medication” on the dashboard -> fills the medication form -> submits -> the medication appears in their list and on the calendar for every date its schedule matches.

ID	Requirement	Priority
REQ-2.1	The system shall allow a patient to create a medication with a name, type, dosage value/unit, and one or more daily times.	Must
REQ-2.2	The system shall support four scheduling modes for a medication: daily, specific days, fixed interval, or as needed.	Must
REQ-2.3	The system shall allow an optional start date and end date to bound the period during which a medication is active.	Should
REQ-2.4	The system shall allow a patient to record optional instructions and prescribing doctor name and phone number.	Could

REQ-2.5	The system shall allow a patient to record optional enable inventory tracking for a medication, recording a current quantity, a refill threshold, and the last refill date.	Could
REQ-2.6	The system shall allow a patient to edit any field of an existing medication they own.	Must
REQ-2.7	The system shall allow a patient to archive (deactivate) a medication rather than permanently delete it, preserving its historical adherence logs.	Must
REQ-2.8	The system shall scope every medication query to the authenticated patient. A patient shall never be able to read or modify another patient's medications directly.	Must

Table 4b- Functional requirements — Medication Management.

4.3. Feature: Daily Schedule and Dose Logging

Description and Priority: Must have. This is the feature the patient interacts with most frequently.

Stimulus/Response sequences: A patient opens the dashboard -> the app computes which medications are due on the selected calendar date based on each medication's frequency rule -> the patient taps a dose's checkbox to mark it taken or skipped -> the log is saved and, if a caregiver is linked, a push notification is sent to that caregiver.

ID	Requirement	Priority
REQ-3.1	The system shall compute, for any selected calendar date, the list of medications and specific dose times due on that date by evaluating each medication's frequency rule against that date.	Must
REQ-3.2	The system shall allow a patient to mark a scheduled dose as "taken" or "skipped", recording the timestamp of the action.	Must

REQ-3.3	The system shall prevent duplicate adherence log entries for the same medication, date, and scheduled time by enforcing a unique composite index and using upsert operation.	Must
REQ-3.4	The system shall decrement a medication's tracked inventory quantity when a dose with inventory tracking enabled is marked as taken	Could
REQ-3.5	The system shall notify every caregiver with an approved link to the patient when a dose is marked taken or skipped, provided the caregiver's device has a registered push token	Should

Table 4c- Functional requirements — Daily Schedule and Dose Logging

4.4. Feature: Automated Reminders and Missed-Dose Detection

Description and Priority: Should have. This feature differentiates MediTracker from a passive logging tool by acting without the patient's input, and was implemented after the core CRUD and logging features were stable.

Stimulus/Response Sequences: A background job runs every minute on the server -> it identifies doses that are due now and sends a reminder, and separately identifies doses that were due more than a 30 minutes grace period ago with no corresponding log entry -> it creates a missed log automatically and notifies the patient and any linked caregivers.

ID	Requirement	Priority
REQ-4.1	The system shall run a scheduled job at a fixed 1-min interval to evaluate all active medications against the current time.	Should
REQ-4.2	The system shall send a push notification to the patient when a scheduled dose time is reached.	Should
REQ-4.3	The system shall automatically create an adherence log entry with status "missed" for any scheduled dose that remains unlogged after a 30-min grace period past its scheduled time.	Should

REQ-4.4	The system shall notify both the patient and any linked caregiver when a dose is automatically marked a missed.	Should
REQ-4.5	The system shall allow a patient to snooze a dose reminder notification.	Could
REQ-4.6	The system shall clear queued/delivered notifications once their corresponding action is resolved, to avoid stale alerts accumulating on the device.	Could

Table 4d- Functional requirements — Automated Reminders and Missed-Dose Detection.

4.5. Feature: Adherence History

Description and Priority: Must have. A log that cannot be reviewed later provides little value to either the patient or the caregiver.

ID	Requirement	Priority
REQ-5.1	The system shall allow a patient to view a chronological history of their own logged doses (taken, skipped, missed).	Must
REQ-5.2	The system shall allow a caregiver to view adherence history of any patient with whom an approved link exists.	Must
REQ-5.3	The system shall prevent a caregiver from viewing the adherence history of a patient with whom no approved link exists.	Must

Table 4e- Functional requirements — Adherence History.

4.6. Feature: Caregiver-Patient Linking

Description and Priority: Must have. The caregiver role provides no value without a defined, consent-based way to connect to a specific patient.

Stimulus/Response sequences: Either a patient or a caregiver search for the other user by email or username and sends an invitation -> the receiving user sees the pending request and approves or rejects it -> once approved, the caregiver gains read access to that specific patient's data, which either party can revoke at any time.

ID	Requirement	Priority
REQ-6.1	The system shall allow a user to send a link invitation to another user identified by email or username, provided the target user has the opposite role.	Must
REQ-6.2	The system shall reject an invitation to oneself or to a user of the same role.	Must
REQ-6.3	The system shall automatically approve a link if both parties have independently sent an invitation to each other.	Could
REQ-6.4	The system shall allow the receiving party of a pending invitation to approve or reject it. The system shall not allow the initiating party to approve their own request.	Must
REQ-6.5	The system shall allow either party of an approved link to revoke it at any time, immediately removing the caregiver's read access to that patient's data.	Must
REQ-6.6	The system shall allow a previously rejected or revoked link to be re-initiated by either party.	Should
REQ-6.7	The system shall enforce a unique constraint per caregiver-patient pair so duplicate link records cannot be created.	Must
REQ-6.8	The system shall let a user view the list of their own links together with each link's current status (pending, accepted, rejected, revoked).	Must

Table 4f- Functional requirements — Caregiver-Patient Linking.

II.5. Other Non-Functional Requirements

5.1. Performance Requirements

ID	Requirement
NFR-1.1	The dashboard's medication list shall load from a single GET request rather than one request per medication, to keep the number of round trips constant regardless of how many medications a patient has.
NFR-1.2	The missed-dose detection job shall complete a full evaluation pass of all active medications within its 1-min execution window so that consecutive runs do not overlap.
NFR-1.3	An adherence log write shall use a single upsert operation rather than a separate find-then-create round trip, to minimize write latency on the most frequently used action in the app.

Table 5a- Non-Functional Requirements — Performance.

5.2. Security Requirements

ID	Requirement
NFR-2.1	All passwords shall be hashed with bcrypt using a salt, the system shall never log, return or store a plaintext password.
NFR-2.2	All protected API routes shall require a valid JWT signed with a serverSide secret. Expired or tampered tokens shall be rejected with a 401 response.
NFR-2.3	Role-based authorization middleware shall enforce that only users with the correct role can reach role-restricted routes.
NFR-2.4	A caregiver shall only be able to read a specific patient's medications or logs after the backend verifies an approved link exists between that caregiver and that patient, checked on every relevant request rather than cached client-side.
NFR-2.5	All communications between client and backend shall occur over HTTPS.

Table 5b- Non-Functional Requirements — Security.

5.3. Usability Requirements

ID	Requirement
NFR-3.1	Marking a dose as taken or skipped shall require a single tap from the dashboard, with no intermediate confirmation screen.
NFR-3.2	The medication form shall only require a name, type, dosage, and at least one schedule time. Every other field shall be optional
NFR-3.3	Role-based navigation shall ensure a user is only ever shown screens and actions relevant to their own role.

Table 5c- Non-Functional Requirements — Usability.

5.4. Reliability and Availability Requirements

ID	Requirement
NFR-4.1	A global Express error-handling middleware shall catch unhandled exceptions and return a structured JSON error response instead of crashing the process.
NFR-4.2	Database connection failures at startup shall be logged clearly to aid diagnosis during deployment.
NFR-4.3	The backend shall be deployed as an always-on web service so that the per-minute cron job for missed-dose detection keeps running continuously.

Table 5d- Non-Functional Requirements — Reliability and Availability.

5.4. Maintainability Requirements

ID	Requirement
NFR-5.1	Backend code shall follow a layered structure separating routes, controllers, middleware, models, and utilities.
NFR-5.2	Mobile screens shall be organized into Expo Router route groups by role.
NFR-5.3	Shared UI elements shall be implemented as a single reusable component rather than duplicated per screen.

Table 5e- Non-Functional Requirements — Maintainability.

II.6. Use Case Diagram and Descriptions

Figure 2 presents the use case diagram for MediTracker. Three actors interact with the system: the patient and the caregiver, both of which are human end users authenticated through the same login flow, and the Cron engine, a system actor representing the backend's automated background process.

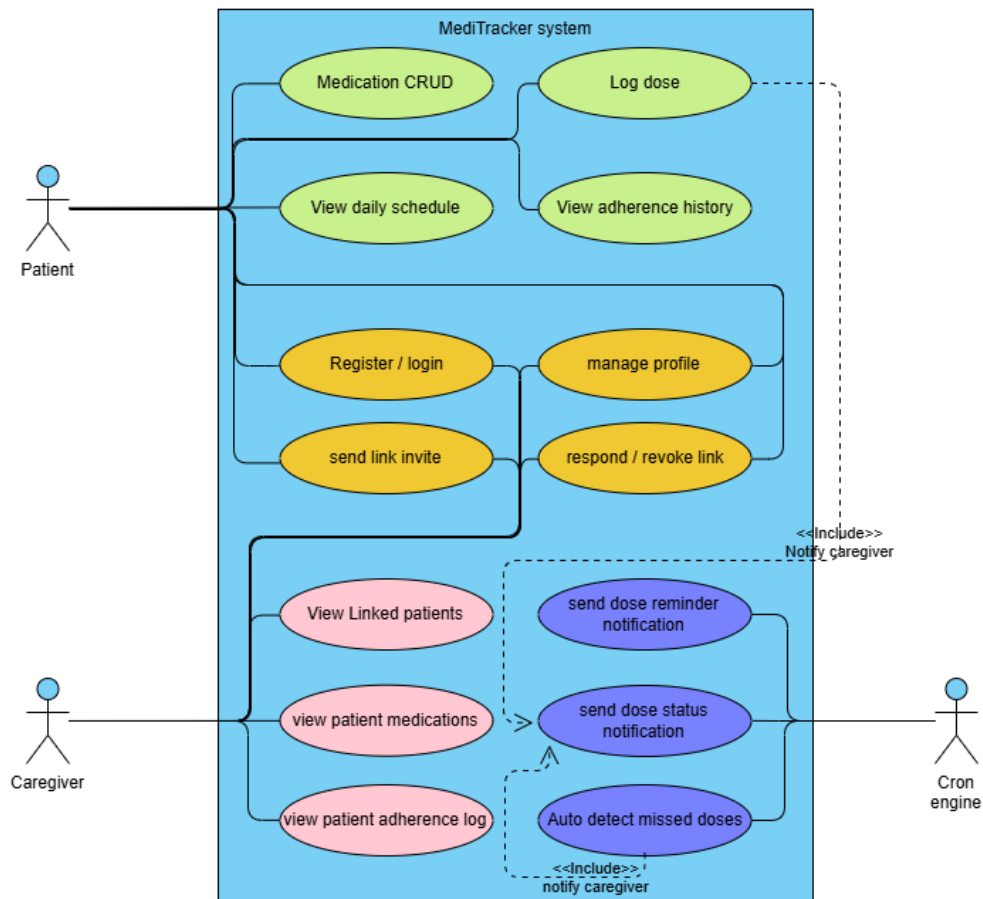


Figure 2- UML use case diagram for MediTracker.

Use Case: Log Dose

Field	Description
Actor	Patient
Precondition	Patient is authenticated and has at least one active medication scheduled for the selected date.
Main flow	1. Patient opens the dashboard and select a date on the calendar. 2. the app lists medications due that day, computed from each medication's frequency rule. 3. patient taps the checkbox next to a dose to mark it taken or skipped. 4. the app send the log to the backend which upserts the adherenceLog entry and if inventory tracking is enabled and the dose was taken, decrement the medication's stock 5. the backend looks up any caregiver with an approved link to the patient and, for each one with a registered push token, sends a notification.
Postcondition	An adherenceLog document exists for that medication/date/time with the chosen status, linked caregivers have been notified.
Alternate flow	If the log already exists for that exact dose slot, the upsert updates it in place rather than creating a duplicate (REQ-3.3)

Table 6a- Use case description — Log Dose

Use Case: Send Caregiver Link Invitation

Field	Description
Actor	Patient or caregiver
Precondition	Actor is authenticated. The target user exists and has the opposite role
Main flow	1. Actor opens the Connections screen and enters the target's identifier. 2. The backend looks up the target and validates the target user. 3. the backend checks for an existing CaregiverLink between the 2 users. 4. if none exists, a new link is created with status pending and initiatedBy set to the actor's role. If a pending link already exists from the target user, it's auto approved. 5. the link becomes visible to both users.
Postcondition	A caregiverLink document exists in a defined state
Alternate flow	If a link already exists in the approved state, the invitation is rejected with an "already exists" message. If it exists as "rejected" or "revoked", it is reopened as pending with the new initiator

Table 6b- Use case description — Send Caregiver Link invitation.

Chapter III — Application Conception

III.1. System Architecture

MediTracker follows a standard three tier architecture, separating the mobile presentation layer, the API business-logic layer, and the database layer. This separation means the mobile client never talks to MongoDB directly, every read or write passes through the Express API, which is the single place where authentication, authorization, and the caregiver-link guard are enforced. Figure 3 illustrates this architecture.

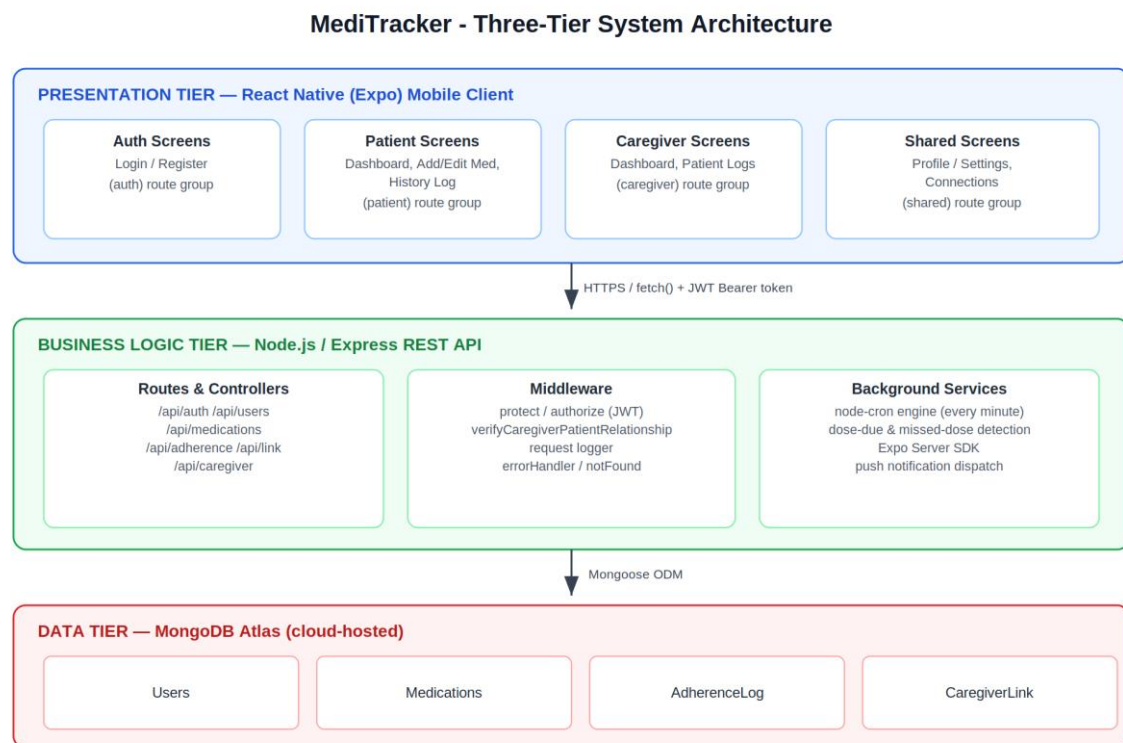


Figure 3- Three-tier system architecture of MediTracker.

The presentation layer is organized into four Expo Router route groups, each corresponding to a layout file that controls what the user is allowed to see based on their role and authentication state. The business logic tier exposes six route modules (auth, users, medications, adherence, link, caregiver), each backed by a controller of the same name, plus a small set of middleware functions applied globally or per-route. The data tier is a single MongoDB atlas cluster holding four collections.

III.2. Database / Data Model Design

MongoDB's document model was used instead of a normalized schema. The table below summarizes the four Mongoose collections. The full attribute list and the relationships between them are shown in the UML class diagram.

Collections	Purpose	Key relationships
Users	Stores both patients and caregivers in a single collection, differentiated by a role field.	Referenced by Medications.user, AdherenceLog.user, and both sides of CaregiverLink.
Medications	One document per medication a patient has registered, including its schedule, dosage, inventory...	Belongs to one patient, generates many AdherenceLog entries.
AdherenceLog	One document per logged or auto-detected event.	References one Medications document and the owning user.
CaregiverLink	One document per caregiver-patient relationship and its current status.	References two Users documents for patient and caregiver.

Table 7-Mongoose collections and their purpose.

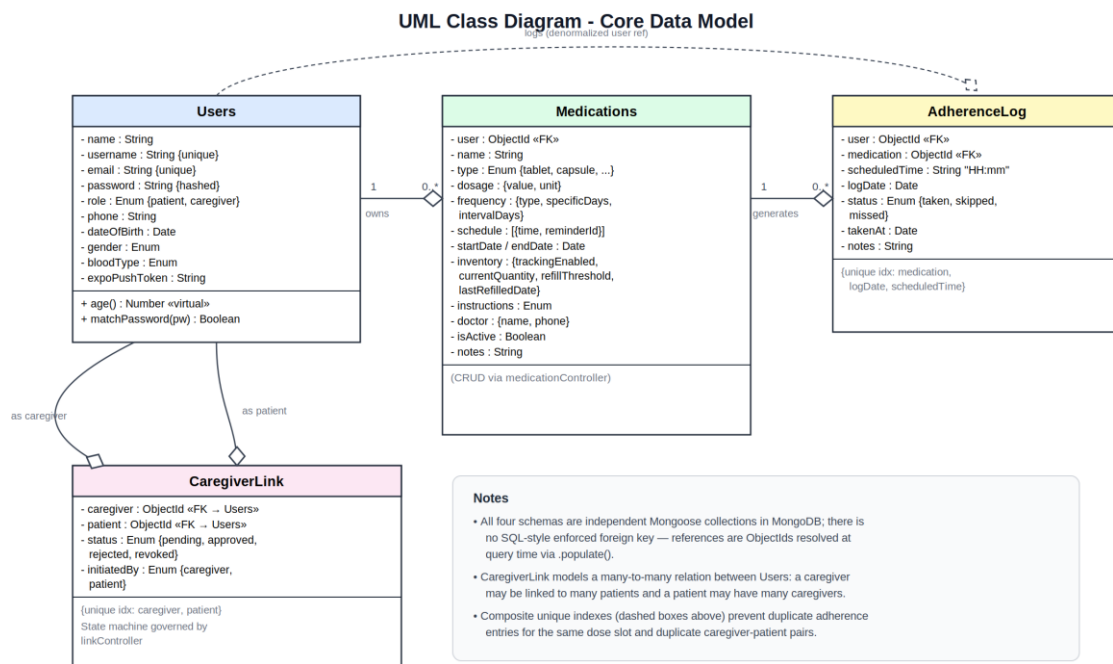


Figure 4- UML class diagram of the core data model.

Two design choices in this model are worth calling out explicitly. First, AdherenceLog stores its own copy of the user reference in addition to the medication reference, this denormalization lets the system query “all of this patient logs” directly without an extra join through Medications, at the cost of having to keep both fields consistent at write time. Second, CaregiverLink is the only collection modeling a relationship between two Users documents rather than between a User and some other entity, it was deliberately kept as its own collection rather than an array of caregiver IDs on the User document, so that the relationship itself could carry state (pending, approved, rejected, revoked) and metadata independently of either user’s own document.

III.3. API Design

The backend exposes a REST API under /api. Every route except registration and login requires a valid JWT. The table below summarizes the endpoint surface as implemented.

Method and Path	Description	Auth
POST /api/auth/register	Create a new user account	Public
POST /api/auth/login	Authenticate and receive a JWT	public
PUT /api/users/changePassword	Update user’s password	JWT
GET /api/users/profile	Get authenticated user’s profile data	JWT
PUT /api/users/updateProfile	Update authenticated user’s profile data	JWT
GET /api/medications	List the patient’s medication	JWT, patient
POST /api/medications	Create a new medication	JWT, patient
PUT /api/medications/:id	Update a medication	JWT, patient
DELETE /api/medications/:id	Archive a medication	JWT, patient
GET /api/adherence/report	List the patient’s adherence history	JWT, patient
POST /api/adherence/log	Create or update a dose log	JWT, patient
POST /api/link/invite	Send a caregiver-patient link invite	JWT

PUT /api/link/respond/:id	Approve or reject a link invitation	JWT
PUT /api/link/revoke/:id	Revoke an approved link	JWT
GET /api/link/mylinks	List all links	JWT
GET /api/caregiver/patientMeds/:id	List linked patient medication	JWT, caregiver, approvedLink
GET /api/caregiver/patientLogs/:id	List linked patient adherence history	JWT, caregiver, approvedLink

Table 8-REST API endpoint summary

III.4. Sequence Diagrams

Two interaction flows were selected for detailed sequence diagrams because they best illustrate the system's two most distinctive behaviors: the automatic caregiver notification, and the stateful resolution logic behind a caregiver-patient link invitation.

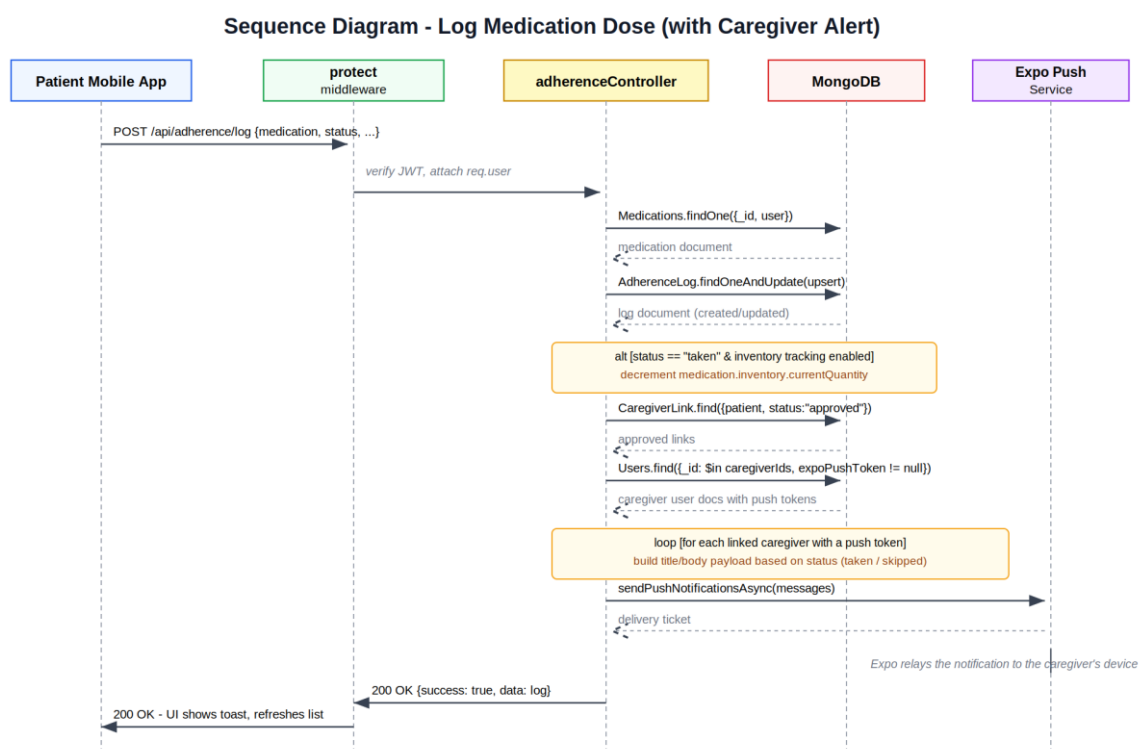


Figure 5- Sequence diagram - log medication dose with caregiver alert

As figure 5 shows, a single client action fans out into three separate concerns on the backend: persisting the log itself, optionally adjusting inventory, and resolving every approved caregiver link to decide who should be notified. Keeping these as separate, sequential steps inside one controller method rather than splitting them into separate API calls, keeps the operation atomic from the client's point of view — a single request either fully succeeds or fails, and the client never has to coordinate three round trips itself.

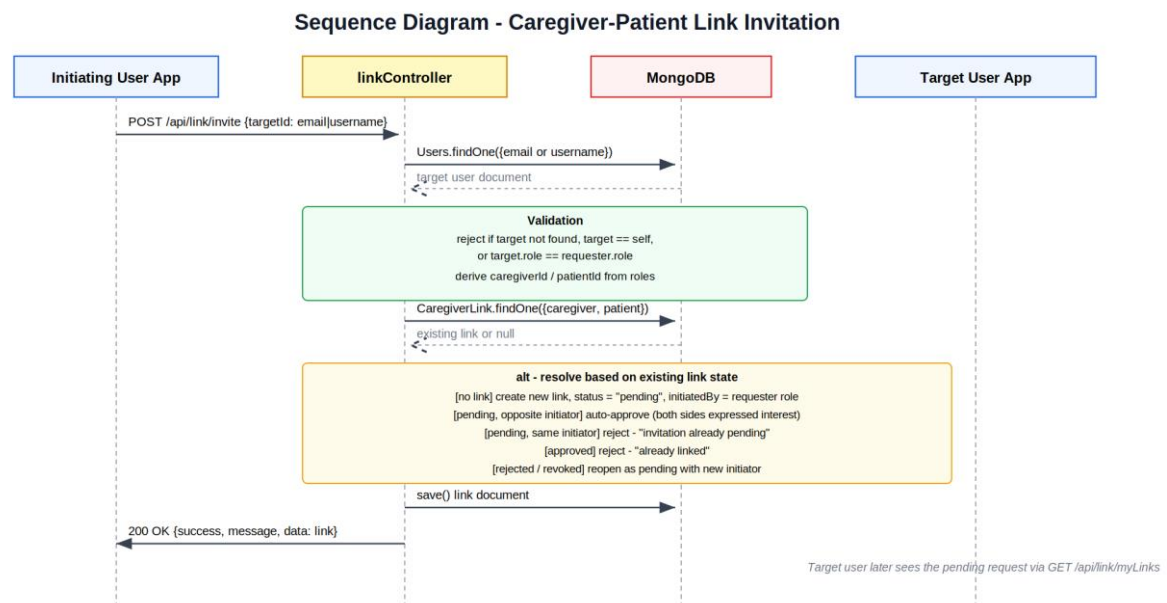


Figure 6- Sequence diagram — Caregiver-Patient link invitation

Figure 6 shows the branching logic that resolves an invitation against whatever link state already exists between the two roles. This logic lives entirely in linkController rather than in the Mongoose schema, which keeps the state-machine rules testable and readable as a single function instead of being scattered across schema hooks.

III.5. State Chart Diagram

The caregiverLink.status field behaves as an explicit state machine rather than a free-form string, with transitions exclusively triggered by linkController methods. Figure 7 documents the full set of states and the guard condition that prevents a user from approving their own invitation.

State Chart Diagram - CaregiverLink Status Lifecycle

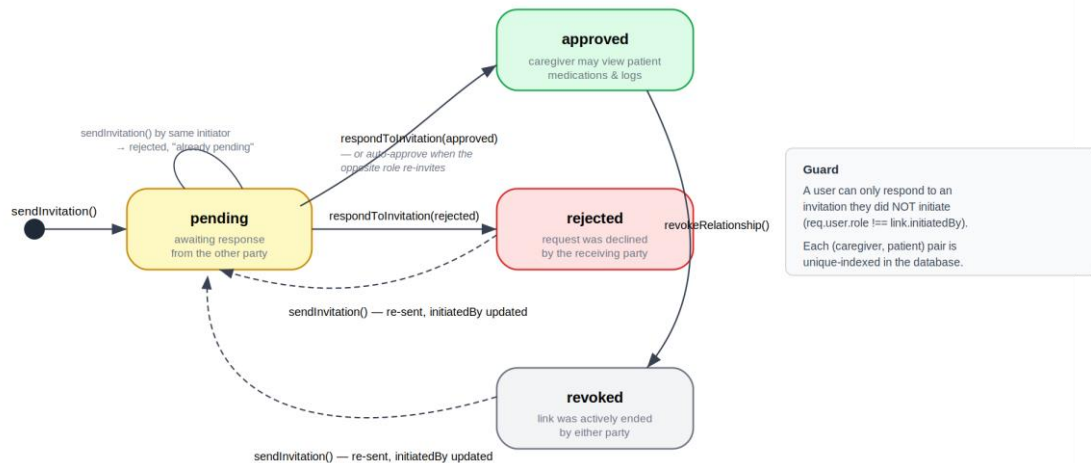


Figure 7- State chart diagram — Caregiver link status lifecycle

Chapter IV — Development, Testing and Deployment

IV.1. Development Methodology

This project was developed solo over approximately four months (Feb to Jun 2026) following an iterative, feature by feature Agile approach rather than a traditional, sequential design-then-build approach. The project's lifecycle was organized into distinct functional iterations, each delivering a vertically complete feature slice — spanning from the database schema and backend API routes up to the frontend user interface components — before progressing to subsequent milestones.

The repository's git architecture utilized fixed main and develop branches for stable releases and staging integration, respectively. Throughout the development lifecycle, short-lived feature branches were provisioned for isolated component creation and subsequently pruned upon successful integration into the development mainline.

The system architecture progressed sequentially through the following core milestones:

- Project scaffolding: Express server, MongoDB connection, the four Mongoose models, and a bare Expo Router structure.
- End-to-End Authentication: JWT generation, password hashing, register/login endpoints, role-based middleware, and the corresponding login/register screens wired to the backend, including persistent auto-login and role-based redirection.

- Medication CRUD: Mongoose methods and protected routes for CRUD operations, followed by the add medication screen, the patient dashboard with a calendar-day selector, and the dashboard's data fetching logic.
- Adherence and Notifications: the adherence controller and routes, taken/skipped checkboxes on the dashboard, a local notification system, the notification schema and backend routes, the automated missed-dose Cron worker, and the history log screen.
- Caregiver role and polish: the user profile screen, the full caregiver frontend and backend, several rounds of bug fixing merged from a dedicated fixBugs branch, and finally switching the backend's deployment target from localhost to a hosted Render service.

This order was not arbitrary, each iteration's feature was a hard prerequisite for the next, notifications are meaningless without the medications to notify about, a caregiver dashboard is meaningless without a patient's data to display, so a strictly sequential, feature-complete-before-moving-on approach reduced the amount of speculative code that might later need to be reworked.

IV.2. Technology stack justification

Layer	Technology	Rationale
Mobile Client	React Native + Expo	Single JavaScript/TypeScript codebase targeting both Android and IOS, Expo's managed workflow provides push notifications, a file-based router, and a fast development loop.
Backend runtime	Node.js + Express	JavaScript on both frontend and backend reduces context switching, Express's middleware model maps directly onto the project's cross-cutting concerns.
Database	MongoDB + Mongoose	The variability of medication's fields map naturally onto MongoDB's flexible document schema.

Authentication	JSON web tokens + bcrypt	Stateless tokens avoid needing a server-side session store, which simplifies horizontal scaling and matches a REST API consumed by a mobile client rather than a browser with cookies
Scheduled jobs	Node-cron	A lightweight, dependency-free way to run a recurring in-progress job without introducing a separate job-queue infrastructure for a project of this scope.
Push notifications	Expo notifications (frontend) + expo-server-sdk (backend)	Both ends of the push pipeline are provided by the same Expo ecosystem already used for the app shell, avoiding a separate firebase admin sdk integration.
Hosting	Render (backend) + MongoDB Atlas (database)	Both offer a free or low-cost tier suitable for an academic project with minimal DevOps overhead

Table 9- Technology stack and justification.

IV.3. Implementation Highlights

Rather than walk through every file in the repository, this section focuses on three pieces of logic that best represent the engineering decisions made during development: the authorization chain that protects caregiver access to patient data, the background reminder engine, and the client-side scheduling logic that mirrors it.

Layered Authorization for Caregiver Access

A caregiver must never be able to read a patient’s medications or logs without an approved link. This is enforced as a chain of two middleware functions applied to caregiver-facing routes: `protect` which verifies the JWT and attaches the requesting user, and `verifyCaregiverPatientRelationship` which checks an approved `caregiverLink` exists for the specific patient ID in the URL. Both run before the controller itself, so the controller code never has to repeat this check.

Background Reminder and Missed-Dose Engine

The most distinctive piece of backend logic is the cron-driven engine that runs once a minute. On each tick it does two independent passes: one looking for medications scheduled at the current minute to send a reminder, and one looking 30 minutes in the past to check whether that earlier dose was ever logged, and if not, to record it as missed and alert both the patient and any linked caregiver.

Client-Side Schedule Computation

The patient dashboard needs to answer the same question as the cron engine — is this medication due on this date? — but on demand, for whatever date the patient has selected on the calendar, rather than only for the current minute. This logic is implemented independently on the client in TypeScript, evaluating the same four frequency types against a selected JavaScript Date.

Keeping this logic on the client side avoids a network round trip every time the patient browses a different day, since the full medication list is already fetched once when the dashboard loads.

Role-Based Route Protection on the Frontend

On the mobile side, each route group's `_layout.tsx` reads the authenticated user's role from a shared `AuthContext` and redirects away from the group entirely if the role does not match. This gives a faster, friendlier failure mode than waiting for a 403 response from the backend, while the backend's own role middleware (`authorize("role")`) remains the actual security boundary, since a client-side redirect can always be bypassed.

IV.4. Challenges Encountered

Several concrete issues surfaced during development, each traceable to a specific commit in the project history. They are listed here rather than smoothed over, since they reflect real engineering trade-offs.

- Redundant data fetching: the patient dashboard was initially calling its medication fetch function up to four times on a single login, caused by overlapping `useEffect`

triggers. This was reduced to two calls by consolidating the triggering conditions, a further reduction was not pursued given the limited time remaining in that iteration.

- Routing and logout edge cases: navigating directly to a nested route or reloading the app on certain screens occasionally left a logged-out user able to briefly see a stale screen before the redirect fired. This was traced to a missing `_layout.tsx` guard in some route groups and fixed by adding the guard consistently to every group.
- Caregiver dashboard flicker and crashes: the caregiver-side screen initially had a visible flicker on the dashboard and in some cases caused the app to crash. Both were addressed in a dedicated round of fixes merged from the `fixBugs` branch before the final submission.

IV.5. Testing and Debugging Approach

No automated test suite was written for this project, verification was performed manually throughout development, which is consistent with the project's scope and timeline as a solo academic project rather than a production system maintained by a team.

Manual testing strategy

- Backend endpoints were exercised directly during development with console logging used to trace request flow and surface errors, including a custom logger middleware.
- The mobile client was tested on Expo Go during early development and on an Expo development build installed on a physical Android device, which was necessary once expo-notifications functionality became the center of the app.
- Each new feature was tested end-to-end as both user roles where applicable.

Debugging process

Bugs were generally found through manual exploratory testing rather than written test cases, then isolated by reproducing the issue, adding targeted console logging around the suspected area, and fixing the root cause before removing or quieting the temporary logging. The global error handling middleware was added early specifically so that an unhandled exception in any route would return a readable JSON error instead of crashing the server outright.

IV.6 Deployment

The backend is deployed as a web service on Render, connected to a MongoDB Atlas cluster, the switch from a localhost backend to this hosted deployment happened in the final iteration, once the feature set was considered complete. The mobile client reads the backend's base URL from a single constants file (`api.ts`) so switching between a local development server and the deployed Render URL required changing one value rather than hunting through every screen.

The mobile application itself was run through Expo's development-build workflow rather than published to the Google Play Store or Apple App Store, distribution for the purposes of this academic project was limited to installing the development build directly on test devices/

IV.7. Conclusion and Future Work

Summary of Achievements

MediTracker delivers a working, end-to-end medication adherence system: secure role-based authentication, full medication CRUD with four distinct scheduling modes, one-tap dose logging with automatic inventory decrement, a genuinely automated missed-dose detection engine that requires no patient input to trigger a caregiver alert, and a complete bidirectional caregiver-linking workflow with explicit state management. Every feature described in Chapter II as "Must have" was implemented and is present in the deployed application.

Limitations

- The app has no offline support: an action taken with no network connection currently fails rather than queueing for later sync.
- The scheduling rule that decides whether a medication is due on a given day implemented independently in two places — the backend cron engine and the mobile dashboard — which is a maintainability risk if the rule ever needs to change.
- The application was developed and tested primarily on Android, IOS specific behavior was not maintained after early development and has not been re-verified, since we can't test an IOS app without an IOS developer license.

Future Considerations

The following were discussed during the project's initial scoping but were not implemented, and remain natural directions for future work rather than claims about the current system:

- Drug-interaction and identification lookups against openFDA API, to warn a patient about known interactions between their tracked medications.
- Offline first data entry on the mobile client, queueing dose logs locally when the device has no connection and syncing once it reconnects.
- A machine learning based adherence risk score, trained on historical logging patterns, to proactively flag patients at higher risk of lapsing rather than only reacting to a single missed dose.
- An Automated test suite and a CI pipeline to run it on every push.
- Consolidating the duplicated schedule-matching logic between the cron engine and the dashboard into a single shared module, callable from both the backend and the frontend via a shared package.